TECHNICAL & USER MANUAL

Digit Studio

手写数字实验室

项目说明书与代码原理详解

从画布到张量，从卷积到预测

从损失到学习，从后台任务到优雅关闭

18 个章节 · 关键代码 · 数学原理 · 操作与维护

0 1 2 3 4
5 6 7 8 9

代码基线　2026-10-11

路径规范　项目相对路径，可迁移至其他电脑

FastAPI + HTML Canvas + PyTorch CNN + MNIST

阅读导航

按使用流程阅读，或点击目录和 PDF 书签跳转到对应章节。文档正文包含关键连续代码摘录，行号与本次基线一致。

快速定位：启动/关闭见第 03、09 章；预处理见第 05 章；CNN 与训练见第 06 至 08 章；接口见第 04 章；故障排查见第 15 章。

本文区分当前实现、教学解释和建议扩展。建议扩展不表示已经具备该功能；源文件遗留注释与真实行为不同时，以函数体和本说明书的核对解释为准。

01 项目全貌与阅读方法 3

02 目录结构、环境与安装 4

03 使用说明与完整操作流程 6

04 系统架构、接口与一次识别的数据流 8

05 图像预处理：从笔迹到张量 11

06 CNN 结构与逐层数学原理 15

07 学习原理：数据、损失和参数更新 18

08 训练代码、保存策略与曲线解读 21

09 多线程、共享状态与生命周期 24

10 前端画布、请求队列与结果呈现 27

11 模型推理、权重文件与设备管理 30

12 特征图、混淆矩阵与可解释性 32

13 离线报告、CSV 与实验记录 34

14 测试方法、证据和覆盖边界 36

15 迁移、命令行实验与故障排查 38

16 二次开发：从需求到可验证改动 40

17 教学实验、术语与常见误区 41

18 代码索引、版本基线与参考资料 43

01 项目全貌与阅读方法

1.1 这个项目解决什么问题

Digit Studio 是一个在本机运行的手写数字实验室。使用者在网页上写一个 0 到 9 的数字，浏览器把画布图像交给 Python 服务，服务经过图像处理和卷积神经网络推理，返回预测类别及十类评分。项目同时提供模型训练和模型探索，用于观察模型如何学习、如何提取图像特征。

识别范围是单张图里的单个数字。网络始终输出 0 到 9 的十类分数，没有字母、汉字、负号、小数、多位数或“未知类别”输出。写入“12”时，模型仍然只能选一个数字；这不是完整 OCR。输入一张照片也不会自动检测照片中的多个数字区域。

项目使用自己的 HTML/CSS/JavaScript 界面，Python 端使用 FastAPI 和 PyTorch。没有前端编译步骤，没有 Node.js 运行依赖，也没有调用远程 AI 识别接口。浏览器只负责采集和呈现，实际 CNN 计算在本机 Python 进程里进行。

1.2 三个工作区

工作区	输入	处理过程	输出与用途
手写识别	320×320 画布、官方测试样本	预处理和一次模型前向传播	预测数字、十类概率、输入预览、处理耗时
模型训练	轮数、批大小、学习率、优化器和调度器	新建模型，遍历数据，反向传播	训练/验证曲线、日志、权重、离线报告
模型探索	当前笔迹、随机样本、上传图片	提取第一组卷积激活，可选评估测试子集	8/16/32 个特征图、网络摘要、混淆矩阵

1.3 文档中的路径如何理解

本文所有文件路径都以“项目根目录”为基准，例如 mnist_web/app.py；不依赖某位使用者的用户名、盘符或安装目录。运行命令前先进入自己下载或解压得到的项目根目录。项目根目录和 mnist_web/ 目录不同：前者包含 .venv/，后者包含 app.py 和 start.bat。

代码摘录保留当前源文件的行号，便于按路径查找；后续修改源文件后行号可能变化，应优先以函数名定位。部分摘录只展示关键连续行，解释覆盖其所在完整函数。说明书中的示意图和公式用于教学，实际行为以对应代码为准。

1.4 建议阅读路线

首次运行：先读第 02、03、15 章，再在网页尝试一个样本。理解原理：按第 05、06、07、08 章阅读，先弄清输入分布，再理解网络和训练。维护项目：重点读第 04、09、10、11、14、16 章。定位代码：查第 18 章的函数索引与源文件指纹。

02 目录结构、环境与安装

2.1 文件职责

项目相对路径	职责	是否应随意删除
mnist_web/app.py	HTTP 路由、模型管理、后台训练、启动和关闭	核心入口，应保留
mnist_web/model.py	MnistCNN 网络、设备选择、权重读写	核心模型，应保留
mnist_web/train.py	单轮训练、评估、训练生成器和 CLI	训练能力，应保留
mnist_web/utils.py	数据划分、预处理、静态报告工具	公共基础，应保留
mnist_web/static/index.html	页面结构、表单、画布和对话框	前端核心，应保留
mnist_web/static/style.css	样式、布局和响应式规则	前端核心，应保留
mnist_web/static/app.js	画布事件、请求队列、结果、曲线和关闭交互	前端核心，应保留
mnist_web/start.bat	Windows 一键启动并自动打开网页	便利入口，Windows 建议保留
mnist_web/requirements.txt	Python 依赖范围	安装依据，应保留
mnist_web/tests/test_api.py	API、训练和生命周期回归测试	维护依据，建议保留
mnist_web/mnist_cnn.pth	当前推理使用的模型权重	删除后需重新训练或恢复备份
mnist_web/data/MNIST/raw/	官方数据集缓存	可重新下载，离线使用时保留
mnist_web/outputs/	训练图表、CSV、界面预览	部分可再生成，注意保留实验结果
.venv/	项目 Python 虚拟环境	可重建，不能当成源码跨机器复制
PROJECT_HANDOFF.md	项目交接说明	阅读辅助

2.2 安装命令与工作目录

下面是 Windows PowerShell 命令，在项目根目录执行。命令显式调用虚拟环境的 Python，因此不要求先执行 activate，也避免安装到另一个 Python 环境。

示例命令 / 字段示例 · powershell

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r mnist_web\requirements.txt
.venv\Scripts\python.exe mnist_web\app.py --open-browser
```

在 macOS/Linux 中，虚拟环境可使用 .venv/bin/python；start.bat 仅适用于 Windows，Python 入口本身可以跨平台运行。项目 README 描述 Python 3.10 至 3.13；依赖清单是版本范围而非锁定文件，新环境能否组合

安装应以包管理器的实际解析结果为准。

示例命令 / 字段示例 · bash

```
python3 -m venv .venv
.venv/bin/python -m pip install -r mnist_web/requirements.txt
.venv/bin/python mnist_web/app.py --open-browser
```

2.3 依赖分别用于什么

torch 提供张量、自动求导、网络层和优化器；torchvision 提供 MNIST 与图像变换；NumPy 处理像素和索引；Pillow 解码、灰度化与缩放图片；FastAPI 处理路由与请求校验；uvicorn 运行 ASGI 服务；Matplotlib 生成离线图表；pandas 导出 CSV；torchinfo 生成结构摘要；httpx 支持测试客户端。

PyTorch 与 torchvision 应配套安装。GPU 不可用时程序回退 CPU，CPU 可以完成识别、训练和报告。需要 NVIDIA GPU 加速时，应按 PyTorch 官方安装选择器安装适合设备的版本，不应直接把别人的 CUDA 安装命令当作通用命令。[R1]

2.4 数据、权重与源码是三种不同资产

源码描述“怎么计算”，数据提供“用什么样本学习”，权重保存“已经学到的参数”。只有源码而没有权重时可以打开页面，但有笔迹的预测请求会报告模型不可用。已有权重但没有数据时通常可以识别自己的笔迹；随机样本、训练和混淆矩阵首次使用时可能需要下载 MNIST。

已有数据会复用缓存。训练数据默认共 60,000 张，其中 5,000 张用作验证、55,000 张参与参数更新；官方 10,000 张测试数据用于评估。[R2] 不需要每次启动重新训练。

03 使用说明与完整操作流程

3.1 启动、访问与后台进程

Windows 双击 mnist_web/start.bat。脚本切换到自身所在目录，优先寻找上一级 .venv/Scripts/python.exe，然后运行 app.py --open-browser。Python 启动服务器，完成启动阶段后默认浏览器自动打开 http://127.0.0.1:7860。

启动窗口代表正在运行的 Python 进程，通常需要保持打开。网页本身不能在服务已停止时凭空启动 Python；刷新已关闭服务的网页只会重新尝试连接。程序没有安装 Windows 服务，也没有配置开机自启。关闭浏览器标签页不等于关闭 Python 服务。

3.2 第一次识别

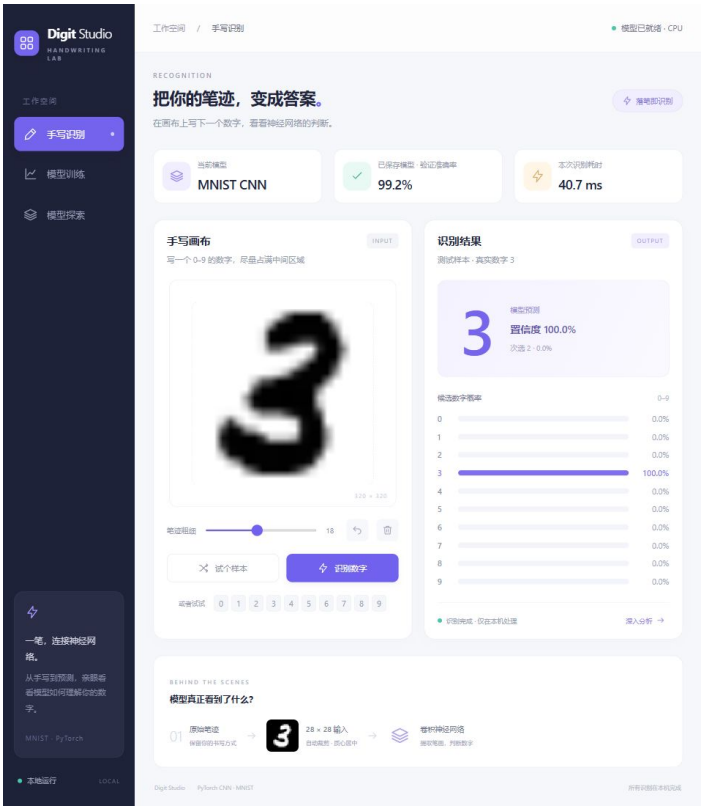


图 1：当前独立网页的识别工作区，画布、候选结果与输入处理流程。

进入“手写识别”，先点击“试个样本”确认网络、数据与模型链路正常，再在画布中央写一个清晰数字。尽量让数字主体占据中间区域，保留边缘空间，不要一次写两个数字。抬笔后会自动识别，也可以点击“识别数字”。

右侧大数字是预测类别，十条概率是模型对 0 到 9 的相对评分。下方 28×28 预览是经过裁剪、缩放后的模型输入显示版，是检查笔迹是否断裂、被误裁或过细的重要工具。“次选”显示第二高分数字；最高概率低于 60% 时前端提示笔迹可能有歧义。这个 60% 是界面提示阈值，不是经过统计验证的拒识阈值。

3.3 笔迹操作

粗细滑块改变下一笔的线宽，不会重绘已有笔迹。撤销恢复绘画或清空等操作前的像素快照，最多保留 20 次。清空恢复白色背景并显示等待输入状态。点击指定数字的快捷按钮会抽取该真实类别的一张随机官方测试样本；真实标签和预测值是不同信息，应分别比较。

概率高也可能预测错误。Softmax 不会检测输入是否符合训练分布，因此字母、涂鸦、照片或多个数字仍可能得到很高的某一类概率。需要区分“模型很自信”和“模型一定正确”。

3.4 训练操作

先备份 mnist_web/mnist_cnn.pth，再打开“模型训练”。默认使用 5 轮、batch size 64、学习率 0.001、Adam、余弦退火。第一次学习训练过程可用 1 轮观察流程；它不保证得到与已有权重同样的准确率。

点击开始后当前浏览器的配置表单禁用，服务端也限制只能运行一个训练任务。曲线在一轮结束并产生记录后更新；准备数据期间没有曲线并不代表程序卡住。训练在后台线程执行，关闭当前标签页不会停止它。新训练创建新网络，不从已有权重继续训练，完成或有效中止后会覆盖当前权重。

点击“停止并保存”是设置停止信号。它不会打断正在执行的张量运算；下一批次边界检测到信号后退出训练循环，之后可能仍会做验证、保存和报告。应等待页面状态完成，不要把“按钮已经点下”理解为“进程已经结束”。

3.5 模型探索

点击“使用当前笔迹”可复制识别页画布；也可随机取样或上传本地图片。上传应是一张清晰、单个数字的图片，尽量让背景一致。选择 8、16 或 32 个特征通道后点击“开始分析”。未选择图片时会自动取一个样本。

勾选混淆矩阵会额外评估 1,000 张测试样本，每类 100 张，比单图分析更耗时。网络结构摘要显示各层形状与参数量。特征图展示第一卷积组的响应，不能单独解释最终类别；详细解释见第 12 章。

3.6 正确关闭与重新启动

点击网页右上角“关闭服务”，确认后页面进入等待状态。服务先请求停止训练，等待训练线程释放锁，再退出 uvicorn。网页停止常规轮询、禁用操作，并显示“服务已关闭”。此时可以关闭网页，下次双击 start.bat 重新启动。

空闲时也可以在启动终端按 Ctrl+C。训练进行中建议优先用网页停止/关闭流程：网页关闭显式等待训练线程，而服务终止阶段只设置 STOP，不能据此保证强制关闭终端或结束进程仍完成保存。断电、任务管理器强制结束进程也不属于优雅关闭。

04 系统架构、接口与一次识别的数据流

4.1 浏览器和 Python 各自负责什么

浏览器负责绘画、表单、导航和结果呈现；Python 负责图片校验、预处理、模型计算、训练和文件保存。页面与接口由同一个 FastAPI 应用提供，因此前端使用 /api/ 开头的相对地址，不需要硬编码某台电脑的 IP，也没有数据库或登录系统。

01 网页画布 / 上传
浏览器：320×320 笔迹或缩小后的图片

↓

02 JSON + PNG Data URL
POST /api/predict 或 /api/analyze

↓

03 解码与预处理
灰度、反色、裁剪、20 像素框、28×28 标准化

↓

04 CNN 前向计算
(1, 1, 28, 28) → logits (1, 10)

↓

05 Softmax 与返回
十类概率、预测数字、PNG 输入预览

↓

06 网页呈现
版本匹配后更新候选条、数字和耗时

4.2 入口与资源路径

源码摘录 · mnist_web/app.py · 第 27 至 39 行

```
27  ROOT = Path(__file__).resolve().parent
28  DEVICE = get_device()
29  MODEL = None
30  META = {}
31  MODEL_LOCK = threading.RLock()
32  JOB_LOCK = threading.Lock()
33  STATE_LOCK = threading.Lock()
34  STOP = threading.Event()
35  CLOSING = threading.Event()
36  LIFECYCLE_LOCK = threading.Lock()
37  TRAINING = {'running': False, 'status': '准备就绪', 'history': {}, 'log': '', 'epochs': 5, 'csv': False}
38  TEST_SET = None
39  DATA_LOCK = threading.Lock()
```

ROOT 由 app.py 的实际文件位置计算，不是当前终端目录。后端再用 ROOT / 'static'、ROOT / 'mnist_cnn.pth'、ROOT / 'data' 等组合路径。因此把整个项目搬到其他目录后，Web 入口仍能找到自己的资源。

注意这一保证主要针对 Web 服务。train.py 的 CLI 默认路径是 ./data、outputs 和 mnist_cnn.pth，会相对当前工作目录解释；使用 CLI 时进入 mnist_web/，或显式提供 --data-dir、--out-dir、--model-path。第 15 章提供不会覆盖现有权重的实验命令。

4.3 HTTP 接口总表

方法	路径	关键输入	成功结果
GET	/	无	静态首页 HTML
GET	/assets/...	静态资源路径	CSS、JavaScript
GET	/api/status	无	模型就绪、设备、参数量、历史验证记录
POST	/api/predict	image、auto_center	blank、prediction、probabilities、input、elapsed_ms
GET	/api/sample	可选 digit=0..9	image、label、index
POST	/api/analyze	image、channels、confusion	预测、features、summary、可选 confusion
POST	/api/train/start	epochs、batch_size、lr、optimizer、scheduler	started=true
POST	/api/train/stop	无必要正文	当前停止状态文字
GET	/api/train/status	无	running、history、log、epochs、csv 等
GET	/api/train/csv	无	训练日志 CSV 文件
POST	/api/shutdown	同源请求	closing=true，表示受理关闭

FastAPI 默认还提供 /docs 和 /openapi.json，可在服务运行时查看交互式接口文档。业务接口没有统一 ResponseModel，因此返回字段由函数构造，修改字段时要同步修改前端和测试。

4.4 请求模型和状态码

源码摘录 · mnist_web/app.py · 第 74 至 90 行

```
74  class ImageRequest(BaseModel):
75      image: str = Field(max_length=6_000_000)
76      auto_center: bool = True
77
78
79  class AnalysisRequest(ImageRequest):
80      channels: Literal[8, 16, 32] = 8
81      confusion: bool = False
82
83
```

```
class TrainingRequest(BaseModel):
    epochs: int = Field(default=5, ge=1, le=30)
    batch_size: Literal[32, 64, 128, 256] = 64
    lr: float = Field(default=.001, ge=.00001, le=.1)
    optimizer: Literal['adam', 'sgd', 'rmsprop'] = 'adam'
    scheduler: Literal['cosine', 'step', 'none'] = 'cosine'

```

Pydantic 在业务函数前做类型和范围校验：训练轮数 1 至 30，批大小只能为 32/64/128/256，学习率为 0.00001 至 0.1；特征数量只能为 8/16/32。网页下拉框提供的学习率范围比接口范围更窄，直接调用 API 时仍会受到服务端检查。

状态码	本项目中的典型含义	处理方式
200	请求成功或任务已受理	根据返回字段更新界面
400	图片无法解码、像素过大、空图分析、digit 越界	修正输入
403	关闭服务请求的 Origin 与服务地址不一致	回到实际本机网页操作
404	不存在的路径、尚无 CSV	检查地址或先完成训练
409	没有模型，或另一个训练已运行	训练/恢复权重，或等待现有任务
422	类型、枚举或数值范围错误	按接口约束修正 JSON
503	正在关闭，或启动方式不支持网页关闭	重启正常入口或使用终端退出

4.5 返回值示例如何理解

示例命令 / 字段示例 · json

```
{
  "blank": false,
  "prediction": 3,
  "probabilities": [0.01, 0.01, 0.02, 0.90, 0.01,
                    0.01, 0.01, 0.01, 0.01, 0.01],
  "input": "<PNG数据>",
  "elapsed_ms": 4.2
}
```

这是字段教学示例，数值不是当前模型一次实际预测记录。probabilities[i] 对应数字 i；prediction 是最大分数的下标。input 是可直接放进 img.src 的 PNG Data URL。elapsed_ms 从后端预处理开始计时，含等待模型锁和推理等开销，但不含此前的图片解码与浏览器网络/渲染时间。

05 图像预处理：从笔迹到张量

5.1 为什么不能直接把画布送进网络

网页画布通常是白底深色笔迹、320×320；MNIST 是 28×28 灰度图，通常黑底浅色数字。尺寸、背景极性、位置、像素量纲不一致，会使模型看到与训练样本完全不同的数据。模型不是按人类方式读取 SVG 或理解“这是一条笔画”，它接收的是数值数组。

因此推理链路先做安全解码，再把图片转换成训练时相同尺度的输入。预处理影响识别结果，不是可以忽略的展示步骤。相同网络权重配上错误反色或错误标准化，很可能输出完全错误但高置信度的结果。

5.2 Data URL 与解码校验

源码摘录 · mnist_web/app.py · 第 92 至 110 行

```
92  def decode_image(data):
93      try:
94          if not data.startswith('data:image/') or ',' not in data:
95              raise ValueError('不是图片数据')
96          raw = base64.b64decode(data.split(',', 1)[1], validate=True)
97          with Image.open(io.BytesIO(raw)) as source:
98              if source.width * source.height > 4_000_000:
99                  raise ValueError('图片过大，请使用小于 400 万像素的图片')
100             # Composite transparent uploads onto white, matching the drawing canvas.
101             rgba = source.convert('RGBA')
102             background = Image.new('RGBA', rgba.size, 'white')
103             background.alpha_composite(rgba)
104             return np.asarray(background.convert('L'))
105     except (ValueError, UnidentifiedImageError, OSError) as exc:
106         raise HTTPException(400, f'无法读取图片：{exc}') from exc
107
108
109 def png_data(array):
110     arr = np.asarray(array)
```

canvas.toDataURL('image/png') 产生带 MIME 前缀的 Base64 文本。后端要求 data:image/ 前缀与逗号分隔，base64.b64decode(validate=True) 校验编码，Pillow 验证实际图片。服务端限制总像素不超过 4,000,000；ImageRequest 还限制 Data URL 字符串长度不超过 6,000,000。

透明 PNG 先合成到白色背景，再转成灰度。这避免透明区域被误解为黑色背景，保持与画板一致。Web 服务并不接受客户端指定任意本地文件路径；输入内容通过 JSON 图片数据传入。工具模块兼容本地路径的函数不等于公开 API 可以读任意文件。

5.3 空白检测的实际含义

源码摘录 · mnist_web/app.py · 第 118 至 132 行

```
118 def infer(array, auto_center=True):
119     if float(np.ptp(array.astype(np.float32))) < 12.75:
```

```
        return {'blank': True, 'probabilities': [0.] * 10, 'prediction': None, 'input': None, 'elapsed_ms'
    : 0}
    started = time.perf_counter()
    display, tensor = preprocess_uploaded_image(array, auto_center=auto_center)
    with MODEL_LOCK, torch.inference_mode():
        if MODEL is None:
            raise HTTPException(409, '没有可用模型，请先训练模型。')
        preds, probs = predict_batch(MODEL, tensor, DEVICE)
    probabilities = probs[0].cpu().tolist()
    return {'blank': False, 'prediction': int(preds.item()), 'probabilities': probabilities,
            'input': png_data(display), 'elapsed_ms': round((time.perf_counter() - started) * 1000, 1)}


@app.get('/')
```

np.ptp 表示最大像素值减最小像素值。动态范围小于 12.75，即 255 的 5%，时返回 blank=true，预测为 None，概率全零，不调用网络。这样清空画布不会被误分类成某个数字。

这是低对比度检测，并不是数字检测。均匀灰色图也会被当成空白；有噪点的纸张可能通过检测；低对比度的真实笔迹也可能被忽略。它解决的是“完全没写时不显示假答案”，不是通用图像内容过滤。

5.4 自动判断反色

源码摘录 · mnist_web/utils.py · 第 600 至 614 行

```
    gray_full = np.asarray(img, dtype=np.float32) / 255.0
    h, w = gray_full.shape
    border = np.concatenate([
        gray_full[0, :].ravel(), gray_full[-1, :].ravel(),
        gray_full[:, 0].ravel(), gray_full[:, -1].ravel()])
    need_invert = invert and float(border.mean()) > 0.5   # 边框偏亮 => 白底 => 需要反色

    # ---- 复用画板的预处理流水线（裁剪 + 质心居中 + 缩放 + 标准化）----
    tensor = preprocess_to_tensor(img, invert=need_invert, auto_center=auto_center)
    display = preprocess_digit(img, invert=need_invert, auto_center=auto_center, apply_normalize=False)
    return display, tensor


def available_test_sample_images(test_set: datasets.MNIST) -> None:
    """占位说明函数：保留接口，实际随机抽样见 sample_test_batch。"""
```

代码读取四条边缘的灰度，平均亮度大于 0.5 时判断背景偏亮，需要反色。反色后数字是高数值、背景是低数值。它适用于边缘大多是纯背景的图片；如果笔迹贴满边缘、照片四周有阴影或背景复杂，边框统计可能判断错误。

_to_uint8_array 处理整数 0..255 和浮点 0..1 两种量纲。浮点最大值不超过约 1 时先乘 255，避免 Pillow 把 0.8 当成 0..255 里的极暗像素。展示值、灰度值和标准化张量必须区分，不应反复归一化。

5.5 裁剪和等比缩放

源码摘录 · mnist_web/utils.py · 第 455 至 475 行

```
    ys, xs = np.nonzero(binary)
    if len(xs) == 0:
        return np.zeros((out_size, out_size), dtype=np.float32)
    y0, y1, x0, x1 = ys.min(), ys.max() + 1, xs.min(), xs.max() + 1
    crop = binary[y0:y1, x0:x1].astype(np.float32)

    # 等比缩放到 20x20 之内（保持长宽比，避免数字被拉变形）
    h, w = crop.shape
    scale = box / max(h, w)
    new_h, new_w = max(1, int(round(h * scale))), max(1, int(round(w * scale)))
    crop_img = Image.fromarray((crop * 255).astype(np.uint8)).resize((new_w, new_h), Image.LANCZOS)
    small = np.asarray(crop_img, dtype=np.float32) / 255.0

    # 按质心对齐到 28x28 画布中心
    canvas = np.zeros((out_size, out_size), dtype=np.float32)
    cy, cx = out_size // 2, out_size // 2
    top, left = cy - new_h // 2, cx - new_w // 2
    canvas[top:top + new_h, left:left + new_w] = small
    return canvas
```

反色后灰度大于 0.12 的像素被保留，其余视为底噪。代码找出非零像素的外接框，按 box/max(h,w) 缩放，使较长边为 20，较短边按同一倍率变化，使用 LANCZOS 重采样。保持比例可以避免把瘦长的 1 拉成方块、把宽数字挤得过窄。

例如外接框高 200、宽 100，缩放因子为 0.1，得到 20×10 的小图；放入 28×28 时，上下约各留 4 像素、左右约各留 9 像素。边界取整会造成一像素差异，属于离散网格的正常结果。

5.6 “质心居中”注释与当前实现的区别

当前 _center_by_mass 这个函数名和部分注释写“按质心居中”，但实际使用 top = 14 - new_h//2 和 left = 14 - new_w//2，按缩放后外接框的几何中心摆放，没有计算像素亮度加权质心。本文按真实代码解释为“外接框居中”。

真正亮度质心应计算 cx = sum(x · I)/sum(I)、cy = sum(y · I)/sum(I)，再平移到目标中心。笔画一侧更重时，几何中心和亮度质心会不同。若以后实现真实质心，还需处理平移边缘裁切，并用同一测试集比较前后效果。当前说明书不会把这个建议描述成已有功能。

5.7 标准化与形状

源码摘录 · mnist_web/utils.py · 第 570 至 579 行

```
    if apply_normalize:
        return ((gray28 - MNIST_MEAN) / MNIST_STD).astype(np.float32)
    return gray28.astype(np.float32)
```

```
575 def preprocess_to_tensor(image, invert: bool = True, auto_center: bool = True) -> torch.Tensor:
576     """预处理并转成可直接送入网络的张量，形状 (1,1,28,28)。"""
577     arr = preprocess_digit(image, invert=invert, auto_center=auto_center, apply_normalize=True)
578     return torch.from_numpy(arr).unsqueeze(0).unsqueeze(0)   # (28,28) -> (1,1,28,28)
579
```

输入先处于 0..1，随后执行 z = (x − 0.1307) / 0.3081。黑背景 x=0 对应约 −0.424，最亮像素 x=1 对应约 2.821，因此最终张量不是 0..1。这里的均值和标准差是代码固定常量，是常见 MNIST 标准化参数；不表示每张单图标准化后严格达到均值 0、方差 1。

28×28 数组通过两次 unsqueeze(0) 变成 (1,1,28,28)。四个维度依次是批次 N、通道 C、高 H、宽 W。单图也必须有批次维度，CNN 层不会把 (28,28) 自动理解成单张单通道图片。

preprocess_uploaded_image 返回两份结果：未标准化的 display 用于网页 PNG 预览，标准化后的 tensor 用于模型。它分别调用预处理函数，因此当前会重复做一次小图处理；数据量很小，但开发者可以在保持结果一致的前提下复用中间结果。

06 CNN 结构与逐层数学原理

6.1 这不是原始 LeNet-5

MnistCNN 借鉴两段卷积/池化加全连接分类的思路，但使用 3×3 卷积、32/64 通道、BatchNorm 和 Dropout，不能把它等同于原始 LeNet-5 的具体层配置。前向实际顺序是 Conv → BatchNorm → ReLU → Pool，不能漏掉 BatchNorm。

源码摘录 · mnist_web/model.py · 第 57 至 69 行

```
57          self.conv1 = nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1)
58          self.bn1 = nn.BatchNorm2d(32)           # 批归一化，加速收敛、稳定训练
59          # conv2: 输入 32 通道 -> 输出 64 通道，卷积核 3x3，padding=1 保证 14x14 不变
60          self.conv2 = nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3, padding=1)
61          self.bn2 = nn.BatchNorm2d(64)
62          self.pool = nn.MaxPool2d(kernel_size=2, stride=2)   # 2x2 最大池化，尺寸减半
63
64          # ---- 全连接分类部分 ----
65          self.fc1 = nn.Linear(64 * 7 * 7, 128)  # 64*7*7 = 3136 -> 128
66          self.dropout = nn.Dropout(dropout)
67          self.fc2 = nn.Linear(128, num_classes)  # 128 -> 10 类 logits
68
69      def forward(self, x: torch.Tensor) -> torch.Tensor:
```

6.2 各层尺寸和可训练参数

阶段	输出形状	可训练参数	含义
输入	(N, 1, 28, 28)	0	单通道灰度批次
conv1	(N, 32, 28, 28)	320	32×(1×3×3+1)
bn1 + ReLU	(N, 32, 28, 28)	64	32 个缩放参数 + 32 个平移参数
第一次 pool	(N, 32, 14, 14)	0	2×2 最大池化
conv2	(N, 64, 14, 14)	18,496	64×(32×3×3+1)
bn2 + ReLU	(N, 64, 14, 14)	128	64 个缩放参数 + 64 个平移参数
第二次 pool	(N, 64, 7, 7)	0	空间再减半
flatten	(N, 3136)	0	64×7×7，保留批次维
fc1 + ReLU	(N, 128)	401,536	3136×128+128
dropout	(N, 128)	0	训练时丢弃概率 0.25
fc2	(N, 10)	1,290	128×10+10
合计	–	421,834	只统计 requires_grad 的参数

BatchNorm 的 running_mean、running_var 和批次计数是缓冲区，会存入 state_dict，但不计入可训练参数。MaxPool 是同一个 self.pool 模块在两处复用，因此摘要里可能出现两次调用；它没有参数，不会导致参数重复学习。

6.3 卷积：局部模式和权重共享

一个 3×3 卷积核在图片不同位置滑动，计算局部像素与权重的加权和。第一层每个输出通道有自己的 3×3 核和一个偏置；同一通道在所有空间位置共享这些权重。训练可能使不同通道对边缘、转角或笔画结构敏感，但不能事先保证某一固定通道一定是“横线检测器”。[R3]

一般空间尺寸公式为 Hout = floor((H + 2P − D·(K−1) − 1)/S + 1)。本项目 K=3、P=1、S=1、D=1，所以 28 保持 28、14 保持 14。padding 在边缘补零，使边缘也能参与类似大小的局部窗口。

第二层卷积每个输出通道都有 32 个输入通道的 3×3 权重，汇总第一层的多种响应。通道数不是图像分辨率；32 个通道表示 32 组特征表示，宽高仍是 28×28。

用一个教学小例子理解加权求和：局部 3×3 像素为 [[0,1,0],[0,1,0],[0,1,0]]，卷积核中间一列权重为 1、其他位置为 0、偏置为 0，则响应为 3；若同样三条亮像素都在左边一列，则这一窗口响应为 0。真实核是训练得到的实数矩阵，不局限于 0/1；这个例子只说明某些局部排列比其他排列更能激活同一个核。

深度学习库中 Conv2d 的具体计算通常是互相关形式，即不把核先翻转再滑动。这不妨碍从训练角度称为卷积，因为核本身会学习。输出一个位置时，先对每个输入通道做局部加权求和，再汇总通道并加偏置；不是让每个输入通道独立得到一个最终类别。

6.3.1 感受野为何逐层扩大

“感受野”表示某个特征位置可能依赖的原始输入区域。以 r 表示范围、j 表示相邻特征点在输入上的间隔，从 r=1、j=1 开始：卷积 3×3 后 r=3，池化 2×2 后 r=4、j=2，第二卷积后 r=8，第二池化后 r=10、j=4。边缘受到 padding 影响，有些依赖来自补零。

位置	理论感受野边长	相邻特征对应的输入间隔
输入像素	1	1
第一次卷积后	3	1
第一次池化后	4	2
第二次卷积后	8	2
第二次池化后	10	4

这说明第二池化输出的单个位置已经整合比原始像素更大的局部上下文，但并非一个位置必然看到整个 28×28 图。展平后的 fc1 同时连接所有空间位置，才把分布在不同位置的特征综合为分类表示。理论感受野不等于所有区域对最终输出贡献一样大。

6.4 BatchNorm 与 ReLU

BatchNorm2d 对每个通道统计批次和空间维上的均值、方差，做归一化并用可训练 gamma、beta 调整。训练模式使用当前批次统计并更新运行统计，评估模式使用保存的运行统计。[R4]

因此 model.eval() 有实质含义，不能把它当成仅关闭梯度。忘记切换到评估模式会使单图预测受当前批次统计影响；忘记 no_grad/inference_mode 则会产生不必要的自动求导开销，两者解决的问题不同。

ReLU(x)=max(0,x)，把负响应置零、正响应保留，为模型加入非线性。若只有线性层连续叠加，整体仍可合并为线性变换；非线性使网络能学习更复杂的类别边界。

6.5 最大池化、展平和全连接

MaxPool2d(2,2) 从每个 2×2 区域选最大响应，空间大小减半，减少后续计算，并让小位置变化对表示的影响有所缓和。池化会丢失精细位置信息，所以它不是免费的准确率提升。

torch.flatten(x,start_dim=1) 把通道和空间维拼成 3136 个特征，保留 N。fc1 将这些特征汇总为 128 维表示，fc2 得到 10 个类别分数。fc1 占约 95.2% 的可训练参数，是模型大小的主要来源；优化参数量时可以优先分析这一层，而不是只调整卷积核。

6.6 Dropout、logits 和 Softmax

Dropout(p=0.25) 在训练时随机将部分激活置零，其余按 1/(1-p) 缩放；评估模式中不随机丢弃。[R5] 它促使网络避免过分依赖少数特征，不是永久删除 25% 的神经元。

源码摘录 · mnist_web/model.py · 第 72 至 85 行

```
72          x = self.pool(F.relu(self.bn1(self.conv1(x))))
73          # 第 2 组：卷积 -> ReLU -> 池化    (N,32,14,14) -> (N,64,7,7)
74          x = self.pool(F.relu(self.bn2(self.conv2(x))))
75          # 展平：  (N,64,7,7) -> (N,3136)
76          x = torch.flatten(x, start_dim=1)
77          # 全连接 + ReLU + Dropout：  (N,3136) -> (N,128)
78          x = self.dropout(F.relu(self.fc1(x)))
79          # 输出层：  (N,128) -> (N,10)，注意这里不接 Softmax，
80          # 因为 CrossEntropyLoss 内部自带 log_softmax（数值更稳定）
81          x = self.fc2(x)
82          return x
83
84
85  def get_device() -> torch.device:
```

最后 fc2 输出 logits，未直接加入 Softmax。推理时 predict_batch 计算 pi = exp(zi)/sum(exp(zj)) 并选择 argmax。数值实现会做稳定处理。因为 Softmax 单调，最大 logits 的下标与最大概率下标相同。

十类概率之和约为 1，浮点误差会造成极小偏差；这不是“模型知道世界只有十个合法输入”。Softmax 只在固定十类里比较分数，不能保证概率经过校准，也不能把 99% 直接解释成现实中有 99% 的正确率。

07 学习原理：数据、损失和参数更新

7.1 训练、验证、测试分别回答什么

训练集用于更新网络参数，验证集用于观察泛化和比较配置，测试集用于最终评估。项目从官方训练数据用固定索引排列切分 5,000 张验证集，其余 55,000 张训练；官方测试集保持独立。

验证数据和训练数据引用相同官方训练集的不同索引，但训练使用带增强的 Dataset，验证使用无增强副本。不能让验证集每次也随机变形，否则曲线变化同时混入数据随机性的影响。

源码摘录 · mnist_web/utils.py · 第 204 至 220 行

```
    g = torch.Generator().manual_seed(SEED)
    perm = torch.randperm(n_train, generator=g).tolist()
    val_idx, train_idx = perm[:val_size], perm[val_size:]
    train_set: Dataset = Subset(train_full, train_idx)
    val_set: Dataset = Subset(val_full, val_idx)

    train_loader = DataLoader(train_set, batch_size=batch_size, shuffle=True,
                              num_workers=num_workers, pin_memory=torch.cuda.is_available(),
                              drop_last=False)
    val_loader = DataLoader(val_set, batch_size=512, shuffle=False,
                            num_workers=num_workers, pin_memory=torch.cuda.is_available())
    test_loader = DataLoader(test_set, batch_size=512, shuffle=False,
                             num_workers=num_workers, pin_memory=torch.cuda.is_available())

    if verbose:
        print(f"[数据] 训练集 {len(train_set)} 张 / 验证集 {len(val_set)} 张 / 测试集 {len(test_set)} 张"
              f" (batch_size={batch_size}，目录={os.path.abspath(data_dir)}) ")
```

固定随机种子 42 提高复现实验能力，但不能保证不同硬件、库版本、后端之间逐位一致。set_seed 在运行时设置 PYTHONHASHSEED，也不能改变进程启动时已确定的全部哈希行为。复现应同时记录版本、设备、划分方式和参数。

7.2 数据增强与 mini-batch

训练变换使用 RandomAffine(degrees=0, translate=(0.08, 0.08))，模拟水平和垂直平移，28 像素尺度约为两像素。没有随机旋转、随机透视或任意缩放。训练 transform 随机平移会使同一样本在不同轮次呈现不同位置，有助于减少对固定位置的依赖。

batch size 64 表示每次拿至多 64 张图计算一份平均损失并更新一次参数；epoch 表示遍历训练集一次。55,000/64 向上取整是 860 批，末批 24 张，因为 drop_last=False。批大小 32/128/256 对应约 1719/430/215 批。批越大更新次数越少，但每次需要更多内存；准确率并不随批大小单调增加。

7.3 交叉熵为何适合十分类

本项目使用 nn.CrossEntropyLoss()，输入是 (N,10) logits，目标是 N 个整数标签 0..9。它内部结合 LogSoftmax 和负对数似然，因此模型最后不能先传入 Softmax 概率再按普通 logits 使用。[R6]

单样本损失 L = -log(py)，其中 y 是真实类别。如果模型给真实类别概率 0.9，损失约 0.105；概率 0.1 时约 2.303。它不仅惩罚预测错误，还会区别“稍微不确定”和“很自信地答错”。默认批次损失是平均值。

对 logits 的梯度可以写成 dL/dzj = pj - 1[j=y]。真实类的概率不足时，梯度推动其分数增加；其他类分数相对降低。实际网络的梯度通过链式法则继续传播到 fc、卷积和 BatchNorm 的参数。

7.4 一次参数更新的五步

源码摘录 · mnist_web/train.py · 第 130 至 138 行

```
130         optimizer.zero_grad(set_to_none=True)   # 清空梯度（set_to_none 更省显存）
131         outputs = model(images)                 # 前向
132         loss = criterion(outputs, labels)       # 计算损失
133         loss.backward()                         # 反向
134         optimizer.step()                        # 更新参数
135         if is_per_batch and scheduler is not None:
136             scheduler.step()
137
138         # ---- 统计 ----
```

先清零是因为 PyTorch 默认会累积梯度；前向计算 logits；损失函数比较 logits 与标签；backward 按计算图反向求导；step 用优化器改变参数。把清零遗漏掉会让本批与旧批梯度叠加，已经改变训练算法。

model.train() 开启训练行为，使 Dropout 和 BatchNorm 使用训练模式；evaluate 和 predict_batch 切到 eval。训练和推理不是同一条执行配置，即使使用同一个网络定义。

7.5 优化器的选择

优化器	当前配置	教学理解	实验注意
Adam	lr 可调，weight_decay=1e-4	结合梯度一阶/二阶滑动统计，自适应调整各参数步长	默认方案；不等于 AdamW
SGD	momentum=0.9，weight_decay=1e-4	沿负梯度方向走，动量平滑更新方向	可能需要不同学习率和更多轮次
RMSprop	lr 可调，weight_decay=1e-4	使用平方梯度的滑动统计调整尺度	不能沿用其他优化器的经验直接保证效果

基本梯度下降形式是 theta_new = theta_old - lr · gradient。学习率太大可能震荡甚至发散，太小可能长时间进步缓慢。weight_decay 约束参数规模，与 Dropout、数据增强一起影响过拟合；它们不是同一个机制。

7.5.1 Adam 的统计量与反向传播再展开

把本批某个参数的梯度记为 gt，Adam 维护一阶统计 mt = beta1 · m(t-1) + (1-beta1) · gt 和二阶统计 vt = beta2 · v(t-1) + (1-beta2) · gt□。初期统计从零开始，使用 mhat = mt/(1-beta1^t)、vhat = vt/(1-beta2^t) 做偏差修正，再按 theta -= lr · mhat/(sqrt(vhat)+epsilon) 更新。本项目未显式覆盖 beta1、beta2、epsilon，使用所安装 PyTorch 的默认值。

这组公式是理解自适应步长的主干；本项目还设置 weight_decay，不能把省略正则项的教学式当作逐位等同的完整实现。SGD 的动量是对更新方向累积，而 BatchNorm 的 momentum 是运行统计更新系数，名字相同但含义不同。

以 fc2 为例，输入特征 h 有 128 维，输出 z=Wh+b。已知 dL/dz 后，可计算 dL/dW=(dL/dz) · h 的外积、dL/db=dL/dz，以及 dL/dh=W□ · (dL/dz)。ReLU 对正值区间的局部导数为 1、负值区间为 0；梯度继续通过 fc1、池化选中的位置、BatchNorm 和卷积传播。PyTorch autograd 自动维护这些依赖，不要求手工写每层梯度。

优化器只修改 model.parameters() 提供的可训练参数，输入图片和标签不是被优化的参数。验证阶段计算 loss 只是统计，没有 backward 和 step，所以验证集不会通过该路径直接改变权重。

7.6 学习率调度器

源码摘录 · mnist_web/train.py · 第 79 至 87 行

```
79      name = (scheduler_name or "cosine").lower()
80      if name == "none" or epochs <= 1:
81          return None, False
82      if name == "step":
83          return torch.optim.lr_scheduler.StepLR(optimizer, step_size=max(1, epochs // 3), gamma=0.3), False
84      # 余弦退火：学习率平滑下降，通常收敛更稳、准确率更高
85      return torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs, eta_min=1e-5), False
86
87
```

CosineAnnealingLR 在总轮数 epochs 范围内按余弦规律从初始学习率向 eta_min=1e-5 下降；StepLR 每 max(1,epochs//3) 轮乘 0.3；none 保持固定。当 epochs<=1 时调度器直接关闭。当前返回 is_per_batch=False，因此每轮验证结束后才 step，而非每个批次更新。

history['lr'] 记录的是 scheduler.step() 之后的学习率，所以它更接近下一轮要使用的值，而不是刚完成这一轮始终使用的值。分析 CSV 时要注意这个时间点，避免把曲线解释错一轮。

08 训练代码、保存策略与曲线解读

8.1 fit 是生成器，不是 HTTP 推送协议

01 开始训练
校验配置 → 获取 JOB_LOCK → 后台线程

↓

02 fit 准备阶段
加载划分数据 → 新建 MnistCNN → 优化器

↓

03 逐批更新参数
zero_grad → forward → loss → backward → step

↓

04 每轮验证与保存
evaluate → scheduler → history → torch.save

↓

05 报告与重载
完整测试集评估 → 图表/CSV → reload_model

↓

06 结束与清理
running=False，清理 STOP，释放 JOB_LOCK

fit 用 yield 把阶段状态交给调用者。app.py 的训练线程循环消费它，更新内存中的 TRAINING；浏览器每 1.5 秒 GET /api/train/status 读取最新快照。yield 不是 WebSocket、SSE 或浏览器流式推送。当前曲线按 epoch 结束时加入历史点；部分旧注释仍写 Plotly 或 batch 实时刷新，但当前前端使用 SVG。

源码摘录 · mnist_web/train.py · 第 287 至 295 行

```
287     def emit(status: str, done: bool = False, stopped: bool = False, log: str = "") -> dict:
288         """统一构造 yield 内容；log 参数用于把多条日志合并成一次输出。"""
289         if log:
290             logs.append(log)
291         return {"status": status, "history": {k: list(v) for k, v in history.items()},
292                 "log": "\n".join(logs), "artifacts": dict(artifacts),
293                 "csv": artifacts.get("history_csv", ""), "done": done, "stopped": stopped}
294
295     # ---------------- 0) 准备数据 ----------------
```

emit 为历史列表建立副本，合并累计日志，并返回 done、stopped、artifacts 和 csv 等字段。yield_every_batch 参数虽然出现在签名中，但当前函数体没有用它控制批次级 yield，不应据此宣称已经具备每批次曲线更新。

8.2 单轮训练与样本加权统计

train_one_epoch 对每批执行参数更新，累计 loss.item()*batch_size，再除以实际处理样本数。这样末批只有 24 张时不会与 64 张大批享有相同统计权重。准确率用正确预测数量/处理样本数量，而不是先算每批准确率再简单平均。

训练准确率是在训练模式、数据增强和 Dropout 条件下计算的；验证准确率在 eval、无增强条件下计算。验证曲线偶尔高于训练曲线并不自动意味着数据泄漏，要考虑这两种条件差异。

例如两个批次分别有 64 和 24 张，平均 Loss 分别为 0.2 和 0.6。简单平均得到 0.4，而按样本加权为 (64×0.2+24×0.6)/88≈0.309。实际训练采用后者。同理，不能把处理了不同样本数的两段准确率不加权地合并。中途停止时分母是实际处理数量，不是计划训练集总数。

8.3 每轮评估、历史和保存

源码摘录 · mnist_web/train.py · 第 371 至 395 行

```
371         val_metrics = evaluate(model, val_loader, criterion, device)
372         if scheduler is not None and not is_per_batch:
373             scheduler.step()                            # 按 epoch 更新学习率
374 
375         epoch_time = time.time() - epoch_start
376         history["train_loss"].append(round(train_metrics["loss"], 6))
377         history["train_acc"].append(round(train_metrics["acc"], 6))
378         history["val_loss"].append(round(val_metrics["loss"], 6))
379         history["val_acc"].append(round(val_metrics["acc"], 6))
380         history["lr"].append(round(optimizer.param_groups[0]["lr"], 8))
381         history["epoch_time"].append(round(epoch_time, 3))
382 
383         # 记录最佳准确率（用于保存"最优模型"）
384         best_acc = max(best_acc, val_metrics["acc"])
385         if save_each_epoch:
386             # 每个 epoch 都保存，网页端"手写预测"可以随时用到最新模型
387             try:
388                 save_model(model, model_path, extra={"epoch": epoch, "best_acc": best_acc,
389                                                      "history": history})
390             except Exception as exc:
391                 print(f"[警告] 模型保存失败：{exc}")
392 
393         stage = "（已被用户停止，保存当前模型）" if stopped_by_user else ""
394         status = (f"□ Epoch {epoch}/{epochs} 完成 {stage} | 训练 loss={train_metrics['loss']:.4f} "
395                   f"acc={train_metrics['acc']:.2%} | 验证 loss={val_metrics['loss']:.4f} "
```

每轮后 evaluate 在验证集上计算损失和准确率，然后更新学习率、追加历史，并更新 best_acc。代码每轮保存当前模型，最终也保存当前模型。best_acc 是截至此时观察到的最高验证准确率，但并没有单独保存“达到最高准确率那一轮的参数副本”。

这意味着当前文件是最后保存的参数，顶部显示的是历史最大验证值，两者不一定对应同一轮。这是现有实现需要明确的边界。如果后续要真正选最优模型，应在验证值提高时另存最佳权重，同时保留最后权重供恢复实验；不能仅修改显示文字当成已解决。

8.4 Web 推理模型什么时候更新

源码摘录 · mnist_web/app.py · 第 201 至 212 行

```
        for update in fit(epochs=config.epochs, batch_size=config.batch_size, lr=config.lr,
                          optimizer_name=config.optimizer, scheduler_name=config.scheduler,
                          num_workers=0, stop_event=STOP, model_path=str(ROOT / 'mnist_cnn.pth'),
                          data_dir=str(ROOT / 'data'), out_dir=str(ROOT / 'outputs')):
            with STATE_LOCK:
                TRAINING.update(status=update['status'], history=update['history'], log=update['log'],
                                csv=bool(update.get('csv')), stopped=update.get('stopped', False))
            if update.get('done') and (ROOT / 'mnist_cnn.pth').exists():
                reload_model()
    except Exception as exc:
        traceback.print_exc()
        with STATE_LOCK:
```

虽然训练会每轮写磁盘，但 Web 训练线程只在收到 done 更新后调用 reload_model。训练进行时识别页通常继续使用此前已经载入内存的 MODEL。新训练结束或有效中止并进入 done 后，才加载当前磁盘文件用于后续识别。

训练模型是 fit 内新建的对象，推理模型是独立载入的对象。这让网页可以继续推理，但它们可能争用 CPU/GPU 资源。不能把“两个对象”理解成“两个任务完全没有性能影响”。

8.5 中途停止的准确语义

每批开头检查 stop_event；当前批次已开始时不会立即中断。中止某一轮后，代码仍对当前模型执行验证、追加这一轮历史，再保存。这一条曲线记录可能对应部分训练集，而不是完整 55,000 张，网页“已完成轮数”实际按历史列表长度显示。

如果 STOP 在进入第一轮之前已设置，历史为空，fit 返回“没有完成任何训练轮次，保留原有模型”，防止随机初始化模型覆盖已有权重。现有测试覆盖这个情形；不能把它扩大解释为任意异常或任意强杀都不会影响权重。

停止信号不贯穿数据下载、全部验证和报告循环。关闭服务可能需要等待这些阶段结束。当前保存使用 torch.save 直接写目标文件，没有临时文件加原子替换机制；需要长期稳健运行时可考虑这一改进。

8.6 如何读 Loss 与 Accuracy

训练 Loss 与验证 Loss 都下降、验证准确率上升，通常表示有效学习。训练继续改善而验证 Loss 上升，可能是过拟合。训练和验证都差，可能是轮数不足、学习率不合适、预处理错误或模型容量不足。单看某一轮波动不能下结论，应看多轮趋势和错误样本。

Loss 和 Accuracy 不一定同向变化：类别预测没变时，真实类概率下降会提高 Loss，却不改变 Accuracy。验证准确率 99% 也不保证手写画板达到 99%，因为画板笔迹与 MNIST 可能存在分布差异。报告实验时要同时说明数据集、样本数、预处理和权重版本。

09 多线程、共享状态与生命周期

9.1 为什么需要后台线程

训练需要很多批次和长时间计算。如果在启动训练的 HTTP 请求里直接完成 fit，浏览器会一直等待响应，也可能阻塞其他任务。当前路由创建 daemon=True 的 threading.Thread，立即返回 started=true，由后续轮询读取进度。

这是单机单进程内的后台作业机制，没有持久任务队列。进程退出后内存状态消失，不会自动恢复正在进行的任务。Python 线程不等于把计算分配到多个独立服务器；PyTorch 原生算子可能释放 GIL，但 CPU/GPU 资源仍是共享的。

9.2 锁与事件分工

对象	保护或表达的内容	关键行为
MODEL_LOCK / RLock	推理模型、元数据和模型访问	推理、特征、模型替换串行保护
JOB_LOCK / Lock	是否有一项训练作业占用	非阻塞获取失败即返回 409
STATE_LOCK / Lock	TRAINING 状态字典	更新与读取快照时保护
DATA_LOCK / Lock	测试集的惰性初始化	避免多请求同时首次加载
LIFECYCLE_LOCK / Lock	开始训练与开始关闭的交界	避免关闭后仍启动新作业
STOP / Event	请求训练停止	线程在批次边界观察
CLOSING / Event	服务已经进入关闭阶段	拒绝新训练、幂等受理关闭

锁保护临界区，Event 表达信号；两者不能互相代替。JOB_LOCK 从训练开始保持到 worker finally 释放，不只是保护一行代码，因此 shutdown 线程可以通过等待这把锁判断作业已完成收尾。

9.3 推理和重载为什么加锁

源码摘录 · mnist_web/app.py · 第 42 至 46 行

```
42  def reload_model():
43      global MODEL, META
44      model, meta = load_model(str(ROOT / 'mnist_cnn.pth'), DEVICE)
45      with MODEL_LOCK:
46          MODEL, META = model, meta
```

reload_model 先构造并载入一个完整新模型，再在 MODEL_LOCK 里替换 MODEL 与 META。这样正常推理不会读取正在逐层填充参数的对象。推理本身在锁中运行，其他模型访问需等待。混淆矩阵也占用模型锁，评估期间识别可能变慢，这是当前实现的性能边界。

training_status 使用 deepcopy，而不是把仍在变化的历史列表直接交给 JSON 序列化。返回快照可以降低序列化过程读到中途变化的风险。线程异常在 worker 的 finally 仍会清理 running 和释放作业锁，防止下一次训练永久被拦截。

9.4 一键启动与就绪检测

源码摘录 · mnist_web/start.bat · 第 10 至 23 行

```
10  cd /d "%~dp0"
11
12  if exist "..\.venv\Scripts\python.exe" (
13      set "PY=..\.venv\Scripts\python.exe"
14      echo [启动] 使用虚拟环境: .venv
15  ) else (
16      set "PY=python"
17      echo [提示] 未找到 .venv，使用系统 Python。若缺少依赖请先运行:
18      echo        python -m venv .venv ^&^& .venv\Scripts\activate ^&^& pip install -r requirements.txt
19  )
20
21  set PYTHONIOENCODING=utf-8
22  "%PY%" app.py --open-browser
23  endlocal
```

cd /d "%~dp0" 同时切换盘符与目录；%~dp0 是批处理自身位置。脚本优先选用根目录虚拟环境，再以 --open-browser 启动。它不后台隐藏服务进程，不注册系统自启。

源码摘录 · mnist_web/app.py · 第 294 至 303 行

```
294 def open_browser_when_ready(server, url, timeout=60):
295     import webbrowser
296     deadline = time.monotonic() + timeout
297     while not server.should_exit and time.monotonic() < deadline:
298         if server.started:
299             webbrowser.open(url)
300             return
301         time.sleep(.1)
302
303
```

浏览器线程观察 server.started，最多等待 60 秒，每 0.1 秒检查一次。uvicorn 完成启动后才调用 webbrowser.open；它用操作系统默认浏览器，而不是固定某个浏览器。没有就绪或启动失败时不会提前打开。这不是检查模型 ready=true：模型加载失败被 lifespan 捕获后，服务仍可以启动，网页可用于重新训练。

9.5 网页关闭如何避免响应被截断

源码摘录 · mnist_web/app.py · 第 270 至 275 行

```
270 def finish_shutdown(server):
271     # Let the current training batch finish and persist weights before exiting.
272     with JOB_LOCK:
273         server.should_exit = True
274
```

```
275
```

源码摘录 · mnist_web/app.py · 第 277 至 294 行

```
277 def shutdown_service(request: Request, background_tasks: BackgroundTasks):
278     origin = request.headers.get('origin')
279     if origin and origin != str(request.base_url).rstrip('/'):
280         raise HTTPException(403, '请从本机服务页面关闭。')
281     server = getattr(app.state, 'server', None)
282     if server is None:
283         raise HTTPException(503, '此启动方式不支持网页关闭，请在终端按 Ctrl+C。')
284     with LIFECYCLE_LOCK:
285         if not CLOSING.is_set():
286             CLOSING.set()
287             STOP.set()
288             # Respond first; waiting for training must not block the browser.
289             background_tasks.add_task(
290                 lambda: threading.Thread(target=finish_shutdown, args=(server,), daemon=True).start())
291     return {'closing': True}
292
293
294 def open_browser_when_ready(server, url, timeout=60):
```

关闭路由先检查 Origin，再设置 CLOSING 和 STOP，通过 FastAPI BackgroundTasks 在响应之后启动等待线程。[R7] 等待线程获取 JOB_LOCK 后设置 server.should_exit=True，交由 uvicorn 正常退出。服务器不是在请求处理过程中直接调用 os._exit，因此浏览器有机会收到 closing=true。

LIFECYCLE_LOCK 使启动训练与设置关闭状态的临界区互斥。重复点击关闭不会反复创建等待线程，closing=true 返回表示“关闭已受理”，不表示端口已经释放。浏览器另行检测服务断开后才显示已关闭。

9.6 支持的启动方式与部署边界

python mnist_web/app.py 会运行 main，建立 app.state.server，因此网页关闭可用。直接用 uvicorn app:app 启动不会执行该 main，app.state.server 不存在，关闭路由明确返回 503，要求终端 Ctrl+C。不要把第三方 ASGI 启动方式与项目入口视为完全等价。

当前只监听 127.0.0.1，不对局域网其他设备开放。Origin 检查是基本的同源限制，未出现 Origin 时仍可由本机工具调用；项目没有身份验证。多 worker 会各自拥有模型、锁和训练状态，因此当前不能简单加 --workers 4；网页关闭也只关联当前进程的 server。

10 前端画布、请求队列与结果呈现

10.1 页面如何组织

index.html 中三个 section 分别代表识别、训练和探索。setPage 修改 hidden 属性和导航的 active/aria-current，更新面包屑；切页不重新下载整个应用，也不重新加载模型。CSS 定义桌面侧栏、工作卡片和窄屏布局；图标使用页面内的 SVG symbol，不依赖远程图标 CDN。

canvas 的内部像素尺寸是 320×320，CSS 展示尺寸可以随屏幕变化。指针事件支持鼠标、触控和手写笔，前端并没有识别手写笔压感或多指绘画。

10.2 坐标换算与指针捕获

源码摘录 · mnist_web/static/app.js · 第 83 至 93 行

```
83  function point(event) { const rect=canvas.getBoundingClientRect(); return {x:(event.clientX-rect.left)*320
        /rect.width,y:(event.clientY-rect.top)*320/rect.height}; }
84  canvas.addEventListener('pointerdown',event=>{
85    if(event.button !== 0 && event.pointerType==='mouse') return;
86    event.preventDefault(); canvas.setPointerCapture(event.pointerId); remember();
87    appState.drawing=true; appState.version++; previousPoint=point(event);
88    ctx.fillStyle='#22263d'; ctx.strokeStyle='#22263d'; ctx.lineWidth=appState.brush; ctx.lineCap='round'; c
        tx.lineJoin='round';
89    ctx.beginPath(); ctx.arc(previousPoint.x,previousPoint.y,appState.brush/2,0,Math.PI*2); ctx.fill();
90    appState.ink=true; $('canvas-guide').hidden=true;
91  });
92  canvas.addEventListener('pointermove',event=>{
93    if(!appState.drawing) return;
```

getBoundingClientRect 给出画布在页面中的显示矩形。用 (clientX-left)*320/rect.width 把显示坐标映射回内部像素；若画布显示成 256 像素宽，显示位置偏移 128 就对应内部 x=160。没有这一步，窄屏缩放后笔迹位置会错位。

setPointerCapture 保证按下后即使指针离开画布，仍可收到结束事件，避免一直停留在 drawing=true。线帽和连接设为 round，使轨迹圆润；按下时先画一个圆点，确保轻点也形成可见笔迹。

10.3 撤销快照的成本与规则

remember 保存 ImageData、ink 状态和结果说明，最多 20 条。320×320×4 字节约 0.39 MiB，一组 20 张原始像素快照约 7.8 MiB，另有对象开销。它是像素级撤销，没有矢量笔画编辑、重做栈或跨刷新持久化。

10.4 防抖、忙碌标志与版本号

源码摘录 · mnist_web/static/app.js · 第 65 至 89 行

```
65  function schedulePrediction() {
66    appState.version++; clearTimeout(predictionTimer);
67    predictionTimer=setTimeout(()=>requestPrediction(),140);
```

```
68  }
69  async function requestPrediction() {
70    clearTimeout(predictionTimer);
71    if(appState.drawing) { appState.pending=true; return; }
72    if(!appState.ink) { renderResult({blank:true,prediction:null,probabilities:Array(10).fill(0)}); return;
        }
73    if(appState.busy) { appState.pending=true; return; }
74    appState.busy=true; appState.pending=false;
75    const version=appState.version;
76    $('prediction-status').textContent='正在识别…';
77    try {
78      const result=await api('predict',{image:canvas.toDataURL('image/png'),auto_center:true});
79      if(version === appState.version) renderResult(result);
80    } catch(error) { if(version === appState.version) { $('prediction-status').textContent='识别失败，请重试
        '; toast(error.message); } }
81    finally { appState.busy=false; if(appState.pending || version !== appState.version) { appState.pending=f
        alse; requestPrediction(); } }
82  }
83  function point(event) { const rect=canvas.getBoundingClientRect(); return {x:(event.clientX-rect.left)*320
      /rect.width,y:(event.clientY-rect.top)*320/rect.height}; }
84  canvas.addEventListener('pointerdown',event=>{
85    if(event.button !== 0 && event.pointerType==='mouse') return;
86    event.preventDefault(); canvas.setPointerCapture(event.pointerId); remember();
87    appState.drawing=true; appState.version++; previousPoint=point(event);
88    ctx.fillStyle='#22263d'; ctx.strokeStyle='#22263d'; ctx.lineWidth=appState.brush; ctx.lineCap='round'; c
        tx.lineJoin='round';
89    ctx.beginPath(); ctx.arc(previousPoint.x,previousPoint.y,appState.brush/2,0,Math.PI*2); ctx.fill();
```

抬笔后等 140 毫秒再发起预测，快速连续操作会重置定时器。busy 表示已有一个请求在飞行，pending 表示有更新需要补发；version 在笔迹变化时递增。发请求时记录版本，返回时只在版本仍匹配时渲染，避免旧图片的慢响应覆盖新结果。

例如请求 A 识别旧笔迹时用户又补了一笔，version 从 8 变 9。A 返回只会完成队列收尾，不显示旧答案；随后请求 B 用当前画布并显示其结果。当前没有主动取消旧后端推理，而是在浏览器丢弃过期结果，这两种机制要区分。

10.5 十类概率与曲线绘制

renderResult 按数字 0..9 建立条形概率行，而不是按得分排序整张列表；最大类被高亮，第二高分用于“次选”。renderChart 用 viewBox 为 640×180 的 SVG 绘制网格、折线和圆点。准确率纵轴固定 0..100%，Loss 纵轴按数据最大值留出 15% 空间。

单个 epoch 时横坐标放在中间，空历史显示占位文字。SVG 可以随卡片大小缩放，避免引入完整图表库。当前曲线没有缩放、框选和多实验叠加功能，扩展这些功能时需判断是否值得引入更重的依赖。

10.6 上传与异步分析的版本保护

前端限制上传文件不超过 10 MiB，并将最长边缩小到最多 800 像素，白底合成后重新编码 PNG。后端仍独立校验，不依赖客户端限制，因为直接 API 调用可以绕过网页。

analysisVersion 在输入图改变时递增。旧分析完成后如版本不一致就不显示，避免刚上传 B 图却出现 A 图结果。特征数量和混淆选项属于请求参数，编辑或重构时也要考虑其过期响应问题。

10.7 关闭交互与连接状态

确认对话框提示影响，取消不发送关闭请求。受理关闭后前端清理训练轮询、状态轮询和预测定时器，增加版本号，设置 `main/aside.inert` 禁用常规交互。每秒检查 `/api/status`；3 秒超时继续等待，其他网络失败视为服务已关闭。

这种判断是本机页面的连接观察，不是进程管理器证明：网络错误也可能来自浏览器环境。实际维护可结合终端退出和端口释放判断。关闭后再打开脚本，再刷新旧页面或使用新自动打开的页面即可恢复。

11 模型推理、权重文件与设备管理

11.1 predict_batch 的推理路径

源码摘录 · mnist_web/train.py · 第 205 至 213 行

```
205 def predict_batch(model: nn.Module, images: torch.Tensor, device: torch.device) -> Tuple[torch.Tensor, tor
    ch.Tensor]:
206     """对一批图像做预测。
207 
208     Returns:
209         (preds, probs) —— preds 形状 (N,), probs 形状 (N,10)
210     """
211     model.eval()
212     outputs = model(images.to(device))
213     probs = torch.softmax(outputs, dim=1)
```

函数把输入移动到所选 device，切换 eval，前向得到 logits，做 Softmax 并返回预测与概率。外层 infer 使用 torch.inference_mode，predict_batch 也有 no_grad 装饰器。这减少推理时自动求导开销；不会重新学习或修改可训练权重。

11.2 state_dict 和元数据

源码摘录 · mnist_web/model.py · 第 129 至 134 行

```
129     payload = {"state_dict": model.state_dict(), "arch": "MnistCNN"}
130     if extra:
131         payload.update(extra)
132     abs_path = os.path.abspath(path)
133     torch.save(payload, abs_path)
134     return abs_path
```

权重文件是 torch.save 保存的字典，含 state_dict、arch 和可选 epoch、best_acc、history。state_dict 记录参数与缓冲区，不等于网络定义代码；加载时仍先构造 MnistCNN 再 load_state_dict。改变层尺寸或类别数后，旧权重通常不能直接加载。

load_model 兼容封装字典与裸 state_dict，使用 map_location 将权重映射到当前设备，再 eval。当前使用 weights_only=False，因此模型文件应来自可信来源，不能把陌生 .pth 当普通图片随意加载。项目没有模型文件上传接口。

11.3 能否从中断点恢复训练

当前 fit 默认构造新模型，Web 没有载入旧模型作为训练起点，也没有从文件恢复优化器或 scheduler 状态。因此“保存过权重”不等于“支持断点续训”。即使在 Python 中把旧 model 参数传给 fit，也仍会新建优化器、重置历史，属于微调起点而不是严格复原之前的训练轨迹。

若要真正断点续训，需要保存并恢复模型、优化器、调度器、epoch、随机状态及数据顺序等，并清楚区分 resume 和 fine-tune。建议保存格式添加版本号，避免长期改模型后无法判断文件结构。

11.4 设备选择与性能

get_device 依次尝试 CUDA、Apple MPS、CPU。支持检测不表示驱动、算子、版本组合一定稳定；CUDA 检测异常会尝试回退。CPU 服务启动时把 PyTorch 线程数限制在 min(4,cpu_count)，对小模型单图推理可减少过多线程调度开销。

模型只有约 42 万参数，但单图处理耗时仍受到图片解码、锁等待、CPU 线程、后端首次调用、GPU 数据传输和报告任务影响。GPU 对大批训练通常更有价值，单张很小图不保证明显更快。评估性能应区分冷启动、预处理、网络传输和纯模型耗时。

12 特征图、混淆矩阵与可解释性

12.1 特征图来自哪里

源码摘录 · mnist_web/app.py · 第 174 至 182 行

```
        features = torch.relu(model.bn1(model.conv1(tensor.to(DEVICE))))[0].cpu().numpy()
        summary = model_summary(model, device=DEVICE)
    maps = []
    for m in features[:data.channels]:
        normalized = (m - m.min()) / (float(np.ptp(m)) + 1e-8)
        maps.append(png_data(normalized))
    result.update(features=maps, summary=summary, confusion=None)
    if data.confusion:
        # Use a balanced, deterministic subset; this is labelled explicitly in the UI.
```

特征是第一组 conv1 → bn1 → ReLU 的输出，尚未经过第一次池化，所以每张为 28×28，共 32 通道。选择 8 或 16 只是显示前 8/16 个通道，不改变模型参数，不表示网络只使用这些通道预测。

每张特征独立用 (m−min)/(max−min+1e−8) 映射到 0..1，再生成 PNG。不同通道各自缩放后，视觉亮度不能直接横向比较真实响应大小；某张看起来很亮，可能只是其自身范围被拉伸。全零或常数响应不会因除零导致 NaN。

12.2 能解释什么，不能解释什么

特征图可帮助观察笔画附近的激活和不同局部响应，是理解卷积的教学工具。它不是最终预测的贡献归因，也不是“网络关注区域”的严格证明。若要研究类别贡献，应另做梯度归因、遮挡实验等，并解释这些方法的局限。

完整网络摘要优先使用 torchinfo；失败时用 forward hook 记录层输出与参数。前向 hook 是诊断机制，不是添加新的训练层。函数最终移除 hook，防止后续推理反复追加收集逻辑。

12.3 网页混淆矩阵的样本选择

源码摘录 · mnist_web/app.py · 第 185 至 200 行

```
        indices = []
        labels = np.asarray(dataset.targets)
        for digit in range(10):
            indices.extend(np.where(labels == digit)[0][:100].tolist())
        loader = DataLoader(Subset(dataset, indices), batch_size=256, shuffle=False)
        with MODEL_LOCK:
            metrics = evaluate(model, loader, torch.nn.CrossEntropyLoss(), DEVICE, collect_predictions=Tru
e)
        matrix = np.zeros((10, 10), dtype=int)
        for actual, predicted in zip(metrics['labels'], metrics['preds']):
            matrix[int(actual), int(predicted)] += 1
        result['confusion'] = {'matrix': matrix.tolist(), 'accuracy': metrics['acc'], 'samples': len(indic
es)}
    return result


```

```
199 def training_worker(config):
200     try:
```

代码对每个数字按数据集原顺序取前 100 个索引，组成固定的 1,000 张均衡子集，不是每次随机抽样。loader 的 batch size 是 256、不打乱。矩阵行是真实类别、列是预测类别：matrix[7][1]=4 表示四张真实 7 被预测为 1。

对角线是正确识别数，非对角线显示具体混淆。因为每行恰好 100 张，各行正确比例直观可比；准确率等于 trace(matrix)/1000。均衡且固定便于界面演示，但不能替代完整 10,000 张官方测试集评估，也不能代表任意实际用户输入分布。

12.4 从混淆到实验假设

若很多 4 误判为 9，可以查看错误样本笔画，判断是字形重叠、尺寸/居中、增强不足还是模型特征不足。先提出可以验证的假设，再用独立数据评估；不要看到一个错例就盲目增加网络层。

单类别召回率可以按 matrix[i][i]/sum(matrix[i]) 计算；精确率按 matrix[i][i]/sum(matrix[:,i]) 计算。两者分母不同：前者问“真实 i 找回了多少”，后者问“预测为 i 的结果有多少是真的”。当前页面主要显示矩阵和总体准确率，没有自动列出全部这些指标。

13 离线报告、CSV 与实验记录

13.1 训练结束会生成什么

项目相对路径	内容	样本范围与含义
mnist_web/outputs/samples_before_train.png	训练前 9 张样本及标签	从训练数据来源抽样预览
mnist_web/outputs/training_curves.png	训练与验证 Loss/Accuracy	history 中的各轮记录
mnist_web/outputs/confusion_matrix.png	10×10 混淆矩阵	完整官方测试集
mnist_web/outputs/predictions_16.png	16 张真实与预测对照	测试集固定种子随机抽样
mnist_web/outputs/error_samples.png	错误预测样本	测试集错误样本，最多 25 张
mnist_web/outputs/training_log.csv	训练数值表	epoch、loss、acc、lr、epoch_time_sec

generate_post_training_report 遍历完整测试集、累积矩阵和预测，随后生成静态图。与网页探索的 1,000 张子集不同，它的结果覆盖整个官方测试集。报告会消费时间与内存，停止或关闭时等待收尾是正常现象。

13.2 CSV 字段和统计口径

源码摘录 · mnist_web/utils.py · 第 698 至 709 行

```
698         df = pd.DataFrame({
699             "epoch": np.arange(1, n + 1),
700             "train_loss": history.get("train_loss", [None] * n),
701             "val_loss": history.get("val_loss", [None] * n),
702             "train_acc": history.get("train_acc", [None] * n),
703             "val_acc": history.get("val_acc", [None] * n),
704             "lr": history.get("lr", [None] * n),
705             "epoch_time_sec": history.get("epoch_time", [None] * n),
706         })
707         os.makedirs(os.path.dirname(os.path.abspath(path)), exist_ok=True)
708         df.to_csv(path, index=False, encoding="utf-8-sig")
709         return os.path.abspath(path)
```

CSV 使用 utf-8-sig，便于 Excel 等工具打开。Accuracy 写成 0..1 的比例而不是字符串百分号，0.9922 表示 99.22%。epoch_time_sec 包含该轮训练和验证等环节，但不含所有结束报告阶段。lr 的时序按第 07 章解释。

CSV 只在已有训练历史时生成。网页 csv=true 是本次作业知道已有日志产物的标志，下载接口则检查实际文件存在；磁盘上可能保留以前的报告，因此不能仅看到一个文件名就断定来自本次训练。

13.3 如何保留可比较实验

当前默认输出名固定，多次训练会覆盖同名文件。做正式实验时应给每次实验独立 `out_dir` 和 `model_path`，记录日期、代码指纹、设备、依赖版本、种子、划分、预处理、配置与权重来源。

避免根据测试集反复挑选参数，再把该测试结果当成完全独立评估。验证集用于迭代，测试集应尽量保留到配置确定后使用。观察画板实际体验可另外收集自写数字测试集，并避免把同一个人的高度相似样本同时放进训练与评估。

14 测试方法、证据和覆盖边界

14.1 如何运行

从项目根目录先进入 mnist_web，再用根目录虚拟环境的 Python 执行 unittest。测试不要求手动启动 Web 服务器，TestClient 在同一进程中运行应用生命周期。

示例命令 / 字段示例 · powershell

```
cd mnist_web
..\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

本文编写时重新运行了当前测试，9 项通过。已有模型被加载用于真实推理；小规模训练写入测试临时目录，不改写当前 mnist_cnn.pth。这项结果证明当前环境下覆盖到的行为正常，不能推断所有操作系统、依赖组合和任意输入均正常。

14.2 九项测试对应的行为

测试名称	验证什么	不应扩大解读为
test_status_and_static_page	首页存在，模型 ready，参数量正确	完整视觉或所有资源测试
test_blank_and_bad_uploads	空白、非法图片、digit 越界	所有恶意图像安全审计
test_predictions_use_real_model	各类样本、概率和、PNG 预览、至少 8/10 正确	对全体用户笔迹 80% 或 99% 准确
test_features_and_balanced_confusion	8 张特征图、结构摘要、每行 100 张	所有通道、可解释性方法正确性
test_training_lock_and_stop	重复训练被拒绝、停止信号、参数范围	所有训练异常恢复
test_cancelling_before_first_epoch_keeps_weights	第一轮前取消不覆盖旧文件	所有中止都保留旧模型
test_real_training_on_small_subset	32 张临时样本实际反向训练并保存	完整训练质量或最终精度
test_shutdown_waits_for_training_and_blocks_new_jobs	等待锁、拒绝新作业、Origin、重复关闭	外部部署或强杀安全
test_browser_opens_only_after_server_ready	就绪后调用浏览器，退出时不打开	每台机器默认浏览器配置正确

14.3 核心测试代码读法

源码摘录 · mnist_web/tests/test_api.py · 第 49 至 72 行

```
49      def test_shutdown_waits_for_training_and_blocks_new_jobs(self):
50          server = SimpleNamespace(should_exit=False)
51          app.JOB_LOCK.acquire()
52          try:
53              with patch.object(app.app.state, 'server', server, create=True):
54                  self.assertEqual(self.client.post('/api/shutdown', headers={'origin':'https://other.exampl
    e'}).status_code, 403)
55                  self.assertFalse(app.CLOSING.is_set())
56                  response = self.client.post('/api/shutdown', headers={'origin':'http://testserver'})
57                  self.assertEqual(response.json(), {'closing':True})
58                  self.assertTrue(app.STOP.is_set())
59                  self.assertFalse(server.should_exit)
60                  self.assertEqual(self.client.post('/api/train/start', json={}).status_code, 503)
61                  self.assertEqual(self.client.post('/api/shutdown').status_code, 200)
62          finally:
63              app.JOB_LOCK.release()
64              deadline = time.monotonic() + 3
65              while not server.should_exit and time.monotonic() < deadline:
66                  time.sleep(.01)
67              app.CLOSING.clear()
68              app.STOP.clear()
69          self.assertTrue(server.should_exit)
70
71      def test_browser_opens_only_after_server_ready(self):
72          server = SimpleNamespace(should_exit=False, started=False)
```

测试使用假的 server 和实际 JOB_LOCK，先人为占用作业锁，调用关闭接口并断言 should_exit 尚未变 true，释放锁后才要求其变 true。这样检查的是“等待收尾再退出”的顺序，而不需要杀掉测试进程。

训练停止测试用受控生成器和 Event 替代长训练，真实训练测试另用 32 张样本确保参数更新路径确实执行。模拟测试与真实计算测试各自覆盖不同问题，不能把全部测试都替换成只检查返回字符串。

14.4 可复现手工验收清单

双击启动脚本，确认就绪后自动打开网页；随机样本与画布识别均有结果；快速补笔不会显示旧结果；清空后回到等待；分析显示特征，矩阵每行共 100；训练期间第二个开始请求被拒绝；停止后日志结束；点击关闭取消时服务仍可用；确认关闭后终端退出，重新启动仍可访问。

可以在浏览器开发者工具 Network 查看接口请求，在 Console 查看脚本错误。浏览器交互和不同屏幕视觉仍需手工检查，API 测试不证明布局、触控、键盘导航或字体渲染都完美。

15 迁移、命令行实验与故障排查

15.1 安全迁移到另一台电脑

复制源码、需要保留的权重与报告，可按需携带 data/ 缓存；在新机器重建 .venv 并安装依赖。不要依赖原机器的盘符、用户名、虚拟环境绝对路径或 GPU。Web 服务按 app.py 自身路径定位资源，跨机器迁移不需要修改源码中的个人目录。

CLI 推荐在 mnist_web/ 下运行；下例把实验权重写入 outputs/，避免覆盖当前模型。命令参数只是教学默认值，不是精度承诺。

示例命令 / 字段示例 · powershell

```
cd mnist_web
..\.venv\Scripts\python.exe train.py --epochs 1 --batch-size 64 --lr 0.001 --model-path outputs/experiment
    -01.pth --out-dir outputs/experiment-01
```

实验输出与当前 Web 权重分离。若确认要使用实验权重，应先停止当前服务、备份原文件，再把选定权重替换为 mnist_cnn.pth 并重新启动；当前没有网页权重版本选择器。

15.2 常见故障按链路定位

现象	优先检查	建议处理
双击窗口闪退	Python 路径、依赖错误	在 PowerShell 中运行 start.bat 查看错误输出
ModuleNotFoundError	调用的 Python 是否来自 .venv	用显式 venv Python 安装 requirements
浏览器连接失败	服务是否运行、端口是否匹配	启动脚本，等待；检查终端错误
端口被占用	旧实例是否还在运行	关闭旧实例，再启动，避免重复开服务
服务启动但无模型	权重是否存在或不匹配	恢复可信权重或重新训练
随机样本/训练失败	数据缓存、联网下载	确认 data/MNIST/raw/ 完整或网络可用
所有输入偏向同一个数字	量纲、反色、标准化、旧权重	检查 28×28 预览和实际预处理
样本正确而自己笔迹不准	画布分布与 MNIST 差异	写清楚、留边距、查看预览和错误样本
训练开始后长时间没有曲线	是否仍在加载或尚未完成一轮	看终端进度；曲线并非每批刷新
GPU 显存不足	批大小、其他占用、包兼容	减少批大小，关闭无关 GPU 任务
网页关闭返回 503	是否直接用 uvicorn CLI 启动	使用 app.py 入口或终端 Ctrl+C
停止/关闭迟迟未结束	是否处于下载、验证或报告阶段	等待收尾，不连续强杀进程
CSV 无下载或内容似旧记录	本轮是否完成报告	比对文件时间和实验目录

15.3 端口修改要同步哪些位置

当前地址集中出现在 main 的 uvicorn.Config、启动提示以及 open_browser_when_ready 的参数中。修改 port 后应同步这些位置及 README；前端 /api/ 相对地址会跟随页面端口，无需改每个 fetch。更稳妥的扩展是把 host/port 定义成统一配置并生成 URL，避免遗漏。

15.4 当前不属于公网产品

本项目没有多用户认证、权限隔离、持久化作业队列、审计日志或 HTTPS 部署方案。仅在本机使用时不需要把这些复杂机制强加给用户；若将监听地址改为 0.0.0.0 对外开放，就必须重新设计训练/关闭控制和资源限制，而不能只改一个 host 当成完整部署。

15.5 注释与实现不一致的维护事项

train.py 顶部和部分 docstring 仍提到 Plotly、逐 batch 推送；utils.py 部分标题和兼容说明还保留 Gradio/Plotly 字样。当前实际前端不使用这些框架。--interactive-plot 虽然存在，但 utils.py 选用 Agg 后端，因此当前不应保证此选项会弹出可交互窗口。

这些遗留文字不影响当前核心 Web 链路，却容易误导维护者。建议后续整理注释、类型注解和未使用参数时另行验证；本次说明书只记录现状，不悄悄改变程序行为。preprocess_uploaded_image 的返回注解写为 NumPy 元组，实际第二项是 torch.Tensor，也是可修正的文档/类型问题。

16 二次开发：从需求到可验证改动

16.1 美化界面该改哪里

结构改 index.html，布局与颜色改 style.css，行为改 app.js。不要只改元素 ID 而不改 JS 的 $('id')，否则页面启动就会报空对象访问。新增按钮时定义可理解的状态：初始、处理中、成功、失败、禁用；异步请求尤其要考虑重复点击和过期响应。

不必为轻量单机应用立即引入前端框架。若功能规模变大，可以再把 app.js 的绘画、API、训练与分析拆成模块；是否引入打包工具应与维护收益匹配。

16.2 识别效果优化的顺序

先建立固定评估集合，分清官方测试效果和真实画板笔迹效果；接着检查反色、裁剪、位置、线宽、缩放与输入预览；再比较真实亮度质心和当前外接框居中；最后才考虑增强策略、网络或训练参数。没有同一评估集的“改完看起来更准”很容易只是抽样偶然性。

若收集自写数据，应保证标注清楚、来源多样、训练与评估隔离。可尝试适度旋转/尺度增强，但数字 6/9 等类的语义会受大角度旋转影响，增强幅度不能无限增加。

16.3 训练功能的改进路线

建议区分 last.pth 和 best.pth，保存真实最佳权重；采用临时文件加替换降低写入中断风险；增加 run_id 与独立输出目录；实现明确的 resume；让 STOP 在验证/报告阶段也有可控响应；如果需要批次进度，通过专门状态通道而非让每批生成整个报告。

这些属于建议扩展，当前没有实现。每个改动都应有能失败的回归测试，例如最佳模型测试需要制造后续验证下降，不能只断言存在一个名叫 best.pth 的文件。

16.4 性能与并发的改进路线

现在混淆评估持有 MODEL_LOCK，可缓存固定子集评估结果，或把不可变模型快照交给独立评估任务；当前推理串行保护可先测等待时间再优化。前端长操作可增加明确的分析进度和取消；训练可记录阶段耗时帮助判断卡顿。

要扩展到多进程或多人共享，应把内存状态和任务管理迁出进程，设计统一锁、共享文件版本及任务恢复。直接增加 worker 数量会绕过当前 JOB_LOCK 的全局单训练假设。

16.5 增加类别或多位数识别

fc2 从 10 改成更多输出还不够：需要新数据、标签映射、权重重新训练、前端概率列表、API schema、混淆矩阵和测试同步调整。多位数还需要分割或检测，或者使用序列识别模型，不能让原十分类网络一次性返回多个数字。

模型输入尺寸变化也会影响 fc1 的 3136 输入维。若改成 32×32，两次池化后为 8×8，64×8×8=4096，旧 fc1 和权重不兼容；可通过自适应池化等方法设计新的网络，但那已是新的模型版本。

17 教学实验、术语与常见误区

17.1 五个可以动手的实验

实验 A：使用同一个数字反复平移到画布不同位置，查看 28×28 预览和预测变化，解释 auto_center 的作用。保持字形不变，避免同时改变多个变量。

实验 B：写一个容易混淆的 4/9 或 1/7，逐笔补充，再比较次选和概率。观察网络是否平滑变化，并验证旧请求不会覆盖补笔后的结果。

实验 C：固定评估数据，分别训练 1 轮和 5 轮，保存到独立目录，比较验证 Loss 与完整测试错误样本。不要仅对比训练准确率，也不要覆盖现有权重后失去基线。

实验 D：同一输入显示 8、16、32 个特征图，比较通道差异，解释逐通道归一化为何妨碍绝对亮度比较。再观察输出类别，说明更多“显示通道”没有改变模型。

实验 E：在临时测试模型上模拟关闭时仍有作业锁，验证 should_exit 等待锁释放；这是理解事件、锁和后台线程的好例子，无须用长训练反复试验。

17.2 术语速查

术语	本项目中的准确含义
Tensor	带形状、类型和设备的数值数组
Channel	一组特征响应，非类别或图像个数
Logits	网络输出的未归一化类别分数
Softmax	把固定类别分数映射为和约为 1 的概率向量
Loss	用于参数学习的连续目标值
Accuracy	预测类别与真实标签一致的比例
Epoch	计划中的一次训练集遍历；中止记录可能不完整
Batch	一次参数更新使用的一组样本
Learning rate	优化器更新步长参数
Overfitting	训练表现好但未见数据泛化较差
Checkpoint	保存的模型状态及可选元数据
Inference	固定权重的前向预测，不更新参数
Event	线程之间传递状态信号的对象
Lock	限制共享资源同时访问的互斥机制

术语	本项目中的准确含义
Data URL	把媒体类型和 Base64 数据放入一个字符串
ASGI	Python Web 应用与服务器之间的接口规范

17.3 常见误区逐一澄清

“99.2% 表示我写什么都准”：它是已保存元数据中的历史验证值，不是个人笔迹准确率。“点击训练是在原来模型上继续变强”：当前从新模型开始。“显示 32 通道比 8 通道识别更准”：显示数量不改变网络。“停止按钮立刻保存退出”：它先发信号，等待后续阶段。“网页关闭就省资源”：关闭标签页不关闭服务。

“Loss 越低就是测试集越准”：训练 Loss 降低可以伴随过拟合。“固定种子就完全一致”：跨环境仍可能不同。“权重文件就是整个应用”：网络定义、预处理、前端与依赖仍不可缺。“名字叫 center_by_mass 就一定是质心”：要阅读函数体。“曲线每 1.5 秒刷新就每批更新”：轮询频率与历史生产频率不同。

17.4 如果要向别人解释这个项目

可用四步说明：网页把笔迹变成 PNG；后端把 PNG 变成与 MNIST 相似的标准化 28×28 张量；CNN 用学习到的卷积和全连接参数得到十类分数；Softmax 和界面把分数呈现为候选结果。训练则通过已知标签和交叉熵反向调整这些参数，验证/测试检查其泛化。

演示时先展示输入预览，再展示概率、特征图和混淆矩阵，最后解释训练曲线。这样观众能先看到数据表示，再理解网络，而不是把“神经网络”当成无法解释的黑盒口号。

18 代码索引、版本基线与参考资料

18.1 如何查找代码

以下索引在 PDF 生成时直接从当前 Python 源文件的 AST 读取，包括顶层函数、类和类方法。行号对应本次文档基线，后续编辑后应以名称为准。

mnist_web/app.py

符号与起始行	职责定位 / 代码类型
reload_model · L42	函数
test_set · L49	函数
lifespan · L58	异步函数
ImageRequest · L74	类定义
AnalysisRequest · L79	类定义
TrainingRequest · L84	类定义
decode_image · L92	函数
png_data · L109	函数
infer · L118	函数
index · L133	函数
status · L138	函数
predict · L148	函数
sample · L153	函数
analyze · L166	函数
training_worker · L199	函数
start_training · L222	函数
launch_training · L229	函数
stop_training · L247	函数
training_status · L257	函数
download_csv · L263	函数
finish_shutdown · L270	函数

符号与起始行	职责定位 / 代码类型
shutdown_service · L277	函数
open_browser_when_ready · L294	函数
main · L304	函数

mnist_web/model.py

符号与起始行	职责定位 / 代码类型
MnistCNN · L40	类定义
MnistCNN.__init__ · L47	类方法
MnistCNN.forward · L69	类方法
get_device · L85	函数
create_model · L112	函数
save_model · L119	函数
load_model · L137	函数
count_parameters · L188	函数

mnist_web/train.py

符号与起始行	职责定位 / 代码类型
build_optimizer · L53	函数
build_scheduler · L72	函数
train_one_epoch · L91	函数
evaluate · L158	函数
predict_batch · L205	函数
show_sample_images · L220	函数
fit · L247	函数
parse_args · L443	函数
main · L460	函数

mnist_web/utils.py

符号与起始行	职责定位 / 代码类型
set_seed · L54	函数
get_device_info · L66	函数
model_summary · L83	函数
get_transforms · L136	函数
get_dataloaders · L156	函数
denormalize · L224	函数
_new_fig · L242	函数
fig_to_array · L249	函数
save_fig · L256	函数
plot_sample_grid · L265	函数
plot_training_curves · L298	函数
plot_confusion_matrix · L329	函数
plot_prediction_grid · L354	函数
plot_error_gallery · L389	函数
plot_feature_maps · L416	函数
_center_by_mass · L448	函数
_to_uint8_array · L476	函数
preprocess_digit · L526	函数
preprocess_to_tensor · L575	函数
preprocess_uploaded_image · L581	函数
available_test_sample_images · L613	函数
sample_test_batch · L618	函数
save_history_csv · L686	函数
generate_post_training_report · L718	函数
format_seconds · L807	函数
now · L814	函数

mnist_web/tests/test_api.py

符号与起始行	职责定位 / 代码类型
test_directory · L21	函数
ApiTests · L32	类定义
ApiTests.setUpClass · L34	类方法 / 测试用例
ApiTests.tearDownClass · L39	类方法 / 测试用例
ApiTests.test_status_and_static_page · L42	类方法 / 测试用例
ApiTests.test_shutdown_waits_for_training_and_blocks_new_jobs · L49	类方法 / 测试用例
ApiTests.test_browser_opens_only_after_server_ready · L71	类方法 / 测试用例
ApiTests.test_blank_and_bad_uploads · L81	类方法 / 测试用例
ApiTests.test_predictions_use_real_model · L91	类方法 / 测试用例
ApiTests.test_features_and_balanced_confusion · L104	类方法 / 测试用例
ApiTests.test_training_lock_and_stop · L115	类方法 / 测试用例
ApiTests.test_cancelling_before_first_epoch_keeps_weights · L137	类方法 / 测试用例
ApiTests.test_real_training_on_small_subset · L152	类方法 / 测试用例

18.2 源码指纹与验证记录

文档生成时记录关键源码的 SHA-256 前 16 位，用于核对说明书和实际项目是否属于同一基线。指纹不是签名，也不证明代码安全，只用于判断文件内容是否变化。

项目相对路径	SHA-256 前 16 位
mnist_web/app.py	616b87793a340c58
mnist_web/model.py	dfdabf1156a15d48
mnist_web/train.py	dcaa0900a2838cd1
mnist_web/utils.py	1c9bbb521635fc9b
mnist_web/static/index.html	be84f1d6eb7ccc0e

项目相对路径	SHA-256 前 16 位
mnist_web/static/style.css	f16667a475cd8a54
mnist_web/static/app.js	f5c66ec312de11b5
mnist_web/start.bat	39eeb7c000ab5a77
mnist_web/requirements.txt	0b02aa09d2b6745c
mnist_web/tests/test_api.py	8444ea7606954026

验证记录：2026-10-11，当前环境运行 unittest discover，9 项通过；未启动长期服务，未对现有权重执行新训练。编写时检查当前 Web 入口、预处理、网络、训练、前端、关闭与测试代码。项目没有完整依赖锁，因此不提供跨环境逐位复现保证。

18.3 官方资料与阅读方向

以下链接用于核对框架 API 和继续学习；说明书的项目行为主要依据本地源代码，不把新版文档的全部功能当成当前已安装库功能。访问核对日期：2026-10-11。

[R1] PyTorch 官方安装选择器：用于选择适合操作系统、计算平台的安装方式。https://pytorch.org/get-started/locally/

[R2] Torchvision MNIST：Dataset 参数、下载与返回值。https://docs.pytorch.org/vision/stable/generated/torchvision.datasets.MNIST.html

[R3] PyTorch Conv2d：卷积参数、输入输出形状。https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html

[R4] PyTorch BatchNorm2d：训练统计、运行统计与评估行为。https://docs.pytorch.org/docs/2.14/generated/torch.nn.BatchNorm2d.html

[R5] PyTorch Dropout：训练时丢弃、缩放与评估行为。https://docs.pytorch.org/docs/2.14/generated/torch.nn.Dropout.html

[R6] PyTorch CrossEntropyLoss：logits 输入和标签约束。https://docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html

[R7] FastAPI BackgroundTasks：响应之后执行后台操作。https://fastapi.tiangolo.com/tutorial/background-tasks/

界面设计思路参考：Numberly 的工作区和候选结果表达，以及 Handwritten Digit Recognition 的独立网页输入方式。当前项目重新实现界面和接口，保留 PyTorch 模型，并非迁移它们的 TensorFlow.js 权重。

https://github.com/soaravant/Numberly

https://github.com/itzdineshx/Handwritten-Digit-Recognition-system

18.4 文档维护约定

修改网络结构时更新第 06、11 章；修改预处理时更新第 05 章和样本验收；修改训练保存/停止时更新第 08、09、13、14 章；修改接口时更新第 04、10 章。重新生成 PDF 后重新检查代码摘录、行号、页面换行与指纹，而不是只替换封面日期。